□□□
□□□□□□ □□□□□□□□□□□□□ □□□□□□□ □□ □□ □□□ □□□□
□□□□□ □□□□□□□□□□□□
□□□

□□□□□□□□□□□□□□ □□□□□□□□ □□ □□ □□□ □□□

□□ □□ □□ □□

□ □□□□ □□ □□ □□□□□□□ □□□ □□□□□□□□□□□□□□□ □□□□□□□ □□□ □□□□ □□□ □□ □□
□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□ □□ □□□□ □□ □□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□ □□□□□□ □□□□ □□□ □□□□□ □□ □□ □□□□□□□□ □□□□□□□□□
□□□□ □□□ □ □□□ □□ □□□□ □□□□□□□□□ □□ □□ □□□ □가□□□

□□ □□ □□□□ □□□□□□□□□ □□ □□□□ □□ □□□□ □□□□ □□□□ □□ □□ □□ □□ □□□
□□□□□□□□□□□□□□□ □□ □□□ □□□□□□□□□□□□□ □□□□□ □□ □□□ □□□□□ □□ □□□ □□□□□
□□□ □□□□ □□ □□ □□□□□□□ □□ □□□ □□ □□□□□□□□□ □□□□□ □□□□□

□□□ □□□□□□ □ □□

□□	□□	□□ □□ □□
□□ □□□	config □□, DataLoader □□, trainer □□	`main.py`
□□	3D CNN □□ PhysNet forward	`neural_methods/model/PhysNet.py`
□□/□□/□□□	train/valid/test loop, checkpoint, metric □□	`neural_methods/trainer/PhysnetTrainer.py`
□□ □□	rPPG□ BVP label □ negative Pearson loss	`neural_methods/loss/PhysNetNegPearsonLoss.py`
□□□ □□/□□□	`.npy` clip loading, face crop, normalization, chunking	`dataset/data_loader/BaseLoader.py`, `dataset/data_loader/UBFCPHYSLoader.py`
□□	HR, SNR, MACC □ □□ □□ □□	`evaluation/metrics.py`, `evaluation/post_process.py`

□□□ □□□ □□

□□□□□□□□□□□□□□□□□□□□□□□□ □□□□ □□□□□□□□ □□ □□□ □□□□ □□□□□

□□	□□
video clip	`[B, 3, 128, 128, 128]`
label / predicted rPPG	`[B, 128]`
frame size	`128 x 128`
clip length	`128`
batch size	config □□ `4`

□□□ □□ □□□□ □□□□ □가 □□□□ □가 □□□□ □□□□ □□□□□ □□□□□ □□□□□□□□ □□□□ □□□□가
□□□□□

□□ □□ □□ □□□ □□

□□□ □□ □□□ □□ □□ □□

□□□□□□□□ □□□□□□□□ □□□□□□□ □□□□□ □□ □□□□□□□ □□□□□ □□ □□□ □□□□□
□□□□□□□□□□ □□□□□□ □□□□□ □□□□□ □ □□□□□ □□□□□ □□□□가 □□□□ □□□□□□
□□□□□□□□□□□ □□□□□□□□□가 □□□□ □가□□□

□□□□ □□□ □□□

```
def forward(self, preds, labels):
    loss = 0
```

```
    for i in range(preds.shape[0]):
        sum_x = torch.sum(preds[i])
        sum_y = torch.sum(labels[i])
        sum_xy = torch.sum(preds[i]*labels[i])
        sum_x2 = torch.sum(torch.pow(preds[i],2))
        sum_y2 = torch.sum(torch.pow(labels[i],2))
        N = preds.shape[1]
        pearson = (N*sum_xy - sum_x*sum_y)/(torch.sqrt((N*sum_x2 -
torch.pow(sum_x,2))*(N*sum_y2 - torch.pow(sum_y,2))))
        loss += 1 - pearson


    loss = loss/preds.shape[0]
    return loss
```

□□□ □□□ □□ □□가 □□□ □□ □□□□□ □□□□□□□□□ □□□□□□□□□ □□ □□□□□□□□□ □□□□□ □□□ □□ □ □□ □□□□ □□□ □ □□□

□□□ □가 □□ □□□□□ □□□□□□ □□

□□ □□□□ □□□ □□ □□□□ □□ □□□□□□□□□□ □□□□□□□□□□□□□□ □□□□□ □□ □□□□ □□□□ □□□가 □□□□□□□□□가ㄱ □□□□□ □□ □□□ □□□□□ □□□□□ □□ □□□ □□ □□□□□□□□□□□□

□□□□ □□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
def _compute_macc(pred_signal, gt_signal):
    pred = deepcopy(pred_signal)
    gt = deepcopy(gt_signal)
    pred = np.squeeze(pred)
    gt = np.squeeze(gt)
    min_len = np.min((len(pred), len(gt)))
    pred = pred[:min_len]
    gt = gt[:min_len]
    lags = np.arange(0, len(pred)-1, 1)
    tlcc_list = []
    for lag in lags:
        cross_corr = np.abs(np.corrcoef(
            pred, np.roll(gt, lag))[0][1])
        tlcc_list.append(cross_corr)
    macc = max(tlcc_list)
    return macc
```

□ □□□ □□□□ □□□□ □□□□□□□□□□□□□□□□ □□ □□□□□ □□□ □□ □□□□ □□ □가 □□□□□ □ □□□□□□□□ □□ □□□□□□□□□ □□□ □ □ □□□

□□□ □□□□□□□□□ □□ □□□□□□ □□□□□□□

□□ □□□□□□□□□□□ □□□ □□□ □□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□□□□□ □□□□□□□ □□□□□ □□□□□□□□□ □□ □□□□가ㅌ □□□□□□ □□□□□□□가ㅌ □ □□□□□□□□□□□□□□□□ □□ □□□ □가□□□ □□ □□□□□

□□□□ □□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

```
def _detrend(input_signal, lambda_value):
    """Detrend PPG signal."""
    signal_length = input_signal.shape[0]
    # observation matrix
    H = np.identity(signal_length)
    ones = np.ones(signal_length)
    minus_twos = -2 * np.ones(signal_length)
    diags_data = np.array([ones, minus_twos, ones])
    diags_index = np.array([0, 1, 2])
```

```
    D = spdiags(diags_data, diags_index,
                (signal_length - 2), signal_length).toarray()
    detrended_signal = np.dot(
        (H - np.linalg.inv(H + (lambda_value ** 2) * np.dot(D.T, D))), input_signal)
    return detrended_signal
```

□□□□ □□□□

```
class VectorizedNegPearson(nn.Module):
    """Batch-vectorized equivalent of PhysNet's negative Pearson loss."""

    def forward(self, preds, labels):
        preds = preds.view(preds.shape[0], -1)
        labels = labels.view(labels.shape[0], -1)

        sum_x = torch.sum(preds, dim=1)
        sum_y = torch.sum(labels, dim=1)
        sum_xy = torch.sum(preds * labels, dim=1)
        sum_x2 = torch.sum(preds.pow(2), dim=1)
        sum_y2 = torch.sum(labels.pow(2), dim=1)

        n = preds.shape[1]
        numerator = n * sum_xy - sum_x * sum_y
        denominator = torch.sqrt(
            (n * sum_x2 - sum_x.pow(2)) * (n * sum_y2 - sum_y.pow(2))
        )
        eps = torch.finfo(preds.dtype).eps
        pearson = numerator / denominator.clamp_min(eps)
        return torch.mean(1 - pearson)
```

□□ □□□ □□□ □□□□□□□ □□□ □□□□□□ □□□□□□□ □□□ □□□□□ □□ □□□□
□□□□□□□□□□□□□□□□□ □□□□□

□□□ □□□□ □□

□□□□ □□□□

```
def fft_circular_macc(pred_signal, gt_signal):
    pred = np.asarray(pred_signal, dtype=np.float64).reshape(-1)
    gt = np.asarray(gt_signal, dtype=np.float64).reshape(-1)
    min_len = min(pred.size, gt.size)
    pred = pred[:min_len]
    gt = gt[:min_len]
    if min_len < 2:
        return float("nan")

    pred_centered = pred - np.mean(pred)
    gt_centered = gt - np.mean(gt)
    pred_norm = np.linalg.norm(pred_centered)
    gt_norm = np.linalg.norm(gt_centered)
    if pred_norm == 0.0 or gt_norm == 0.0:
        return float("nan")

    corr = scipy.fft.ifft(scipy.fft.fft(pred_centered) *
np.conj(scipy.fft.fft(gt_centered))).real
    corr = np.abs(corr / (pred_norm * gt_norm))
    return float(np.max(corr[: min_len - 1]))
```

□□□ □□□ □□□□□ □□□□□□□□□□□□□□□□ □□□ □□ □□□가ㅌ□ □□□ □□가ㅅ□ □□□□□ □□□
□□□□□□□□□□□ □□ □□ □□□가ㅌ□ 가ㅅ□ □□□ □□□ □ □□□

□□□ □□□□□□□ □□

□□□□ □□□□

```
@lru_cache(maxsize=32)
def _detrend_projection(signal_length, lambda_value):
    signal_length = int(signal_length)
```

```
    lambda_value = float(lambda_value)
    h = np.identity(signal_length)
    ones = np.ones(signal_length)
    minus_twos = -2 * np.ones(signal_length)
    diags_data = np.array([ones, minus_twos, ones])
    diags_index = np.array([0, 1, 2])
    d_mat = spdiags(diags_data, diags_index, (signal_length - 2), signal_length).toarray()
    return h - np.linalg.inv(h + (lambda_value**2) * np.dot(d_mat.T, d_mat))


def cached_detrend(input_signal, lambda_value):
    input_signal = np.asarray(input_signal)
    projection = _detrend_projection(input_signal.shape[0], float(lambda_value))
    return np.dot(projection, input_signal)
```

□ □ □ 가ㅌ □ 가ㅌ □ □ □ □ □ □ □
□ □ □ □ □ □ □가 □ □ □ □ □ □가 □ □ □

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

□□	□□□ □□
□□ `PhysNetNegPearsonLoss.py` □□ □□	□□ □□ □□ □□ □ □□ □□□ □□
`PhysnetTrainer` class □□	□□ □□ □□□□□ □□□ □□
`BaseLoader.chunk` generator streaming	raw preprocessing□ □□ □□ □□□ □□
□□ UBFC-PHYS □□□ benchmark	□□ □□□ □□□ □□ □□ □□
AMP □□ `torch.compile`	GPU □□□ □□ □□ □□ □□ □□

□ □ □ □ □ □ □

□ □ □ □ □ □ □

□ □

□□	□
□□ □□	warm-up 1□ □ □ □□ 10□
random seed	100
□□ □□	`time.perf_counter`
peak memory □□	`tracemalloc`
OS	Linux 5.15.0-139-generic
Python	3.8.20
NumPy	1.22.0
SciPy	1.5.2
PyTorch	2.1.2+cu121
device	CPU
□□ □□□ □□ □□	□□□□ □□

□

□□	Before □□±□□□□ ms	After □□±□□□□ ms	□□ □□	Correct
B=4,T=128	0.1521 ± 0.0344	0.0447 ± 0.0036	3.40x	True
B=16,T=128	0.5448 ± 0.0150	0.0438 ± 0.0028	12.43x	True
B=64,T=128	2.1442 ± 0.0119	0.0469 ± 0.0030	45.75x	True
B=64,T=512	2.4575 ± 0.5431	0.1104 ± 0.0133	22.26x	True

□ □ □ □ □ □ □ □ □가 □가 □ □ □ □ □ □ □ □ □ □ □ □ □ □
□ □ □ □ □

□ □ □ □ □ □ □ □ □

□□	Before □□±□□□□ ms	After □□±□□□□ ms	□□ □□	Correct

T=128	8.3623 ± 0.0768	0.0570 ± 0.0062	146.63x	True
T=256	16.9391 ± 0.0944	0.0614 ± 0.0064	275.88x	True
T=512	34.9527 ± 0.1442	0.0665 ± 0.0066	525.86x	True
T=1024	70.3462 ± 0.4283	0.0758 ± 0.0067	927.87x	True

□□ □□가 □가□□□ □□ □□□ □□□□ □□ □□□□□□□□ □□ □□□□□□□□□□□□ □□□ □□ □□□□ □□□□□ □□□□□

□□□ □□□□□□□ □□

□□	Before □□±□□□□ ms	After □□±□□□□ ms	□□ □□	Correct
T=32	0.2115 ± 0.0175	0.0055 ± 0.0008	38.70x	True
T=64	130.5220 ± 64.6791	0.0061 ± 0.0014	21300.83x	True
T=128	596.5495 ± 55.7484	2.4590 ± 5.8683	242.60x	True
T=256	1775.3165 ± 111.4675	0.0189 ± 0.0025	93906.19x	True

□□□ □□□□ □□□□□□ □□

□□	□□	Before peak KiB	After peak KiB
loss	B=64,T=128	1.182	0.938
MACC	T=1024	82.648	64.730
detrend	T=256	2055.391	2.094

□□□□□□□□ □□□□□□가ㅁ □가 □□ □□□ □□□ □□□□□ □□□□□□□□ □□□□□□□□ □□ □□□□□□□ □□□□ □□□□□□ □□□□□□□□□□ □□□□□□□ □□□□□□ □□□□□

□□□ □□ □□ □□

□□	Before	After
negative Pearson	batch loop□ □□ □□ □□□ □□ □□	`VectorizedNegPearson`□□ □□ □□
MACC	lag □□□ corrcoef □□□ □ □□□ □□	FFT □□ □□ □□□ □□
detrend	projection matrix □□□ □□□ □ □□ □□	projection □□ cache□ □□ □□ □□
benchmark	□□ □□ □□	CSV□ □□ JSON □□ □□

□□ □□ □□ □ □□

□□□ □ □□□□가

□□□□□□□□ □□□□□□□ □□□□□ □□□□□□ □□□□□ □□□□ □□□□□□ □□□□ □□□□ □□□ □□□□□ □□□□□□ □□□□가 □ □□ □□□□ □□□□□□□□□가 □□ □□□□□

□□□□□ □□ □□□ □□가 □□□□ □□□ □□□□□ □□□□□ □□ □□□□□□ □□□ □□ □□□ □□ □□□□□□□ □□□□ □□□□□□□□□□□□ □ □□ □□□□ □□□ □□□□□□ □□□□□□가 □□□□ □□□□□□ □□□ □□□□

□□□□□□□ □□□□ □□□□□□ □□□□□□□ □□□□□□□ □□□ □□□□□□□ □□□□□□가ㅅ □ □□□□□ □□□□□ □□□가 □□□□□ □ □□ □□□□□□ □□□□□□□□ □□□ □ □□ □□□□ □□□

□□□ □□ □□□□ □□가 □가

□□	□□□ □ □□
VectorizedNegPearson	batch size□ □ □
FFT MACC	□□ signal/window □□□ □ □
cached detrend	□□ window length□ □□□ □

□□□ □□□□□□□□□□

□□□ □□□□□ □□ □□□ □□□ □□□□□□□□ □□□□□□□□□□ □□□ □□□□ □□□□□ □□□□□□□□

```
physnet-advanced-python-final/
├── README.md
├── requirements.txt
├── src/
│   ├── before/
│   │   ├── PhysNetNegPearsonLoss_before.py
│   │   └── post_process_before.py
│   └── after/
│       └── physnet_optimizations.py
├── benchmark/
│   └── run_benchmark.py
├── results/
│   ├── benchmark_results.csv
│   └── environment.json
└── report/
    ├── report.md
    └── report.pdf
```